PART I — FINAL CAPSTONE PROJECT PROPOSAL & PROTOTYPING

SmartStudy Assistant (SSA): Adaptive Learning & Self-Assessment Platform

Developer: Md. Imran Hosen | **Implementation Stack:** Java (OOP & Swing Prototype) / Web Architecture | **GitHub:** github.com/IMRAN-8/smartstudy-assistant

1. Project Overview & Problem Statement

Students frequently struggle to identify which specific sub-concepts they misunderstand when studying complex course topics. Conventional study tools separate learning into disconnected steps: students read static notes or PDFs in one application, take generic quizzes in another, and manually track review flashcards in a third. This fragmentation fails to link quiz errors directly to targeted re-teaching or long-term memory retention schedules.

SmartStudy Assistant (SSA) unifies five learning stages into one closed-loop system: **Explain** → **Examine** → **Diagnose** → **Re-Teach** → **Spaced Recall**. Learners input a study topic or upload a PDF document, review a structured explanation, take a concept-tagged diagnostic quiz, receive automated remedial micro-lessons for any weak sub-concepts (score below 70%), and review due concepts using the SuperMemo-2 (SM-2) spaced repetition algorithm.

> **Core Objectives:** (1) Parse study topics or PDF documents into structured overviews and worked examples; (2) Generate concept-tagged MCQ and short-answer diagnostic quizzes; (3) Grade submissions via hybrid deterministic and keyword/semantic evaluation; (4) Generate remedial micro-lessons for weak sub-concepts; and (5) Schedule review intervals using the SM-2 algorithm.

2. Software Process Model, Milestones & Gantt Chart

This project uses the **Iterative and Incremental Software Process Model**. Each stage of the five-part learning pipeline is built and verified as a modular increment before integrating the full system.

Phase	Milestone	Key Deliverables	Timeline
Phase 1	M1: Requirements & Proposal	Problem statement, scope definition, process model, and initial proposal	Weeks 1–2
Phase 2	M2: System & UML Modeling	DFD, Use Case, Activity, ERD, Class Diagram, and Object Diagram	Weeks 3–4
Phase 3	M3: 5-Class Java Prototype	5 core domain classes in Java wired to an interactive Swing GUI	Weeks 5–6
Phase 4	M4: Core Adaptive Engine	Hybrid grading logic, weak-concept classifier, and SM-2 scheduler	Weeks 7–8
Phase 5	M5: Testing & Final Delivery	End-to-end verification, documentation, and GitHub release	Weeks 9–10

2.1 Project Completion Gantt Chart (10-Week Schedule)

Task / Work Package	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
1. Proposal & Requirement Analysis	■	■								
2. UML Diagrams & Database Design			■	■						
3. 5-Class Java OOP & GUI Prototype					■	■				
4. Hybrid Grader & SM-2 Integration							■	■		
5. System Verification & GitHub Release									■	■

3. UML & Architectural Modeling Specifications

The six required architectural models below define the data flow, actor interactions, control logic, database entities, object-oriented class blueprint, and runtime object state of SmartStudy Assistant.

3.1 Data Flow Diagram (DFD — Level 1 Pipeline)

Process Node	Input Data Stream	Processing Transformation	Output / Data Store
P1.0 Ingest Input	Topic prompt or PDF file	Normalizes text and initializes session	`D1: study_sessions`
P2.0 Explain Topic	Session context from D1	Generates overview, examples, and summary	`D1: explanations`
P3.0 Build Exam	Structured lesson content	Creates concept-tagged MCQs & short items	`D2: questions`
P4.0 Grade & Diagnose	Learner answers + answer keys	Scores responses; flags concepts below 0.70	`D3: weak_concepts`
P5.0 Re-Teach & SM-2	Diagnosed weak concept tags	Builds micro-lessons & sets SM-2 due dates	`D4: recall_schedules`

3.2 Use Case Diagram Specification

Use Case ID & Name	Primary Actor	UML Relationship	Functional Description
UC-01: Start Study Session	Learner	Includes `UC-02`	Enters a topic or uploads a PDF to generate a lesson.
UC-02: Take Diagnostic Quiz	Learner	Includes `UC-03`	Submits answers to concept-tagged MCQ and short items.
UC-03: Grade & Diagnose Gaps	Grading Engine	Extends to `UC-04` if score < 0.70	Computes scores per concept and classifies severity.
UC-04: Study Remedial Lesson	Learner / AI Engine	Includes `UC-05`	Delivers targeted micro-lessons for weak sub-concepts.

Use Case ID & Name	Primary Actor	UML Relationship	Functional Description
UC-05: Review Spaced Recall	Learner	Updates SM-2 Queue	Rates recall quality (0–5) to calculate next review date.

3.3 Activity Diagram (Control Flow States)

1. **Session Initialization:** Learner authenticates workspace → chooses input mode (**Decision:** *Topic Prompt* vs. *PDF Upload*) → system generates and displays structured study explanation.
2. **Diagnostic Assessment & Hybrid Grading:** System presents concept-tagged questions → learner submits responses → system executes deterministic MCQ check and keyword/semantic short-answer evaluation in parallel.
3. **Mastery Branching:** System evaluates sub-concept averages (**Decision:** *Score ≥ 0.70* vs. *Score < 0.70*). If all concepts ≥ 0.70, session completes as mastered.
4. **Remediation & SM-2 Scheduling:** For every concept < 0.70, system classifies severity (`HIGH` if < 0.40; `MEDIUM` if 0.40–0.69) → generates remedial micro-lesson → initializes SM-2 schedule (1-day or 2-day interval) → updates ease factor and interval upon recall review.

3.4 Entity-Relationship Diagram (ERD Schema)

Entity / Table	Primary & Foreign Keys	Core Attributes	Cardinality
`STUDY_SESSIONS`	PK: `id` | FK: `user_id`	`topic`, `source_type`, `created_at`	1 : 1 with `EXPLANATIONS`
`QUESTIONS`	PK: `id` | FK: `session_id`	`concept_tag`, `question_type`, `prompt`, `expected_answer`	N : 1 with `STUDY_SESSIONS`
`SUBMISSIONS`	PK: `id` | FK: `session_id`	`overall_score`, `submitted_at`	N : 1 with `STUDY_SESSIONS`
`WEAK_CONCEPTS`	PK: `id` | FK: `session_id`	`concept_tag`, `severity`, `average_score`, `micro_lesson`	1 : 1 with `RECALL_SCHEDULES`
`RECALL_SCHEDULES`	PK: `id` | FK: `weak_concept_id`	`ease_factor`, `interval_days`, `repetitions`, `due_at`	1 : 1 with `WEAK_CONCEPTS`

3.5 UML Class Diagram & 5-Class Java Prototype Architecture

The prototype implements five domain classes in Java alongside a Swing graphical interface (`SmartStudyAssistantApp`) that executes the full diagnostic and SM-2 workflow in memory.

Java Class Name	Attributes (Fields)	Key Methods	UML Association
`1. StudySession`	`sessionId`, `userId`, `topic`, `explanationSummary`, `questions: List<ConceptQuestion>`	`addQuestion(q)`, `getQuestions()`, `getTopic()`	Composes `1..*` `ConceptQuestion`
`2. ConceptQuestion`	`position`, `conceptTag`, `questionType`, `prompt`, `expectedAnswer`	`evaluateResponse(ans): double`, `getConceptTag()`	Belongs to `StudySession`
`3. DiagnosticSubmission`	`submissionId`, `session`, `questionScores: Map`, `overallPercentage`	`gradeResponses(list)`, `diagnoseWeakConcepts()`	Instantiates `WeakConceptRemediation`
`4. WeakConceptRemediation`	`conceptTag`, `severity`, `conceptScore`, `microLesson`, `workedExample`	`generateRemedialContent()`, `createInitialRecallSchedule()`	Creates `1` `SM2RecallSchedule`
`5. SM2RecallSchedule`	`conceptTag`, `easeFactor`, `intervalDays`, `repetitions`, `dueDate`	`applySM2Review(quality)`, `getDueDate()`	Tracks longitudinal review state

3.6 UML Object Diagram (Runtime Snapshot — Photosynthesis Test Case)

During runtime verification on the topic *Photosynthesis*, the active object graph holds the following state immediately after grading a learner submission:

- `session_01 : StudySession` — `sessionId = "sess-24060-01"`, `topic = "Photosynthesis & Cellular Energy"`, linked to 4 `ConceptQuestion` instances.
- `q2 : ConceptQuestion` — `position = 2`, `conceptTag = "Calvin Cycle"`, `questionType = "SHORT"`, linked to `sub_01 : DiagnosticSubmission` (`overallPercentage = 31.3%`).
- `weak_01 : WeakConceptRemediation` — `conceptTag = "Calvin Cycle"`, `conceptScore = 0.25`, `severity = "HIGH"`, linked to `recall_01 : SM2RecallSchedule` (`easeFactor = 2.50`, `intervalDays = 1`, `repetitions = 0`).